

Synopsis
on

A MULTI-CLOUD INFRASTRUCTURE-AS-CODE FRAMEWORK FOR AUTOMATED PROVISIONING OF POLYGLOT WEB APPLICATIONS USING OPENTOFU

SUBMITTED TOWARDS PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE OF

MASTER OF TECHNOLOGY

(Computer Science & Engineering)

SUBMITTED BY

Rohit Dalal

Roll No. 2537008

Under the supervision of

Dr. Tarun Dalal

(Department of Computer Science & Engineering)



University Institute of Engineering and Technology
(Maharshi Dayanand University, Rohtak)

Aug-Dec, 2026

1. Title

A Multi-Cloud Infrastructure-as-Code Framework for Automated Provisioning of Polyglot Web Applications Using OpenTofu

2. Introduction

Cloud computing has become the foundation of modern software delivery, with organizations increasingly adopting multi-cloud strategies to improve resilience, control cost, retain negotiating leverage with vendors, and avoid long-term lock-in to a single provider. However, provisioning and maintaining consistent infrastructure across heterogeneous cloud platforms is inherently difficult: each provider exposes different APIs, terminology, and managed services for conceptually identical building blocks - for example, a private network is called a VPC on AWS, GCP, and DigitalOcean, but a Virtual Network (VNet) on Azure, and each platform's managed MySQL and load-balancing offerings differ in capability, default configuration, and cost structure.

Infrastructure as Code (IaC) has emerged as the standard approach to this problem, allowing infrastructure to be defined declaratively, version-controlled, and provisioned repeatably. Terraform popularized this model, but following HashiCorp's 2023 license change away from a fully open-source license, the developer community founded OpenTofu - a Linux Foundation-governed, open-source fork that preserves the same declarative language and provider ecosystem while guaranteeing the tool remains open. OpenTofu's provider-plugin architecture makes it particularly well suited to expressing the same logical architecture across multiple, otherwise incompatible, cloud platforms.

This project presents the design and implementation of a multi-cloud infrastructure automation framework built on OpenTofu. The framework provisions an identical logical architecture - three polyglot application tiers (Node.js, Python, and PHP) placed behind a load balancer with path-based routing, and backed by a managed MySQL database and a managed Redis-compatible cache - independently on four major cloud providers: DigitalOcean, Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure. Each provider's implementation uses that platform's native networking, compute, load-balancing, and managed-database

services, so the underlying resource graph differs from provider to provider even though the resulting application architecture is the same.

The relevance of this work is twofold. First, it functions as a practical reference implementation for teams evaluating a cloud provider, planning a migration, or designing a genuinely portable deployment strategy, since it makes the concrete differences between providers - in networking model, load-balancer capability, database provisioning behaviour, and cost - explicit rather than theoretical. Second, over the course of development the framework was iteratively deployed end-to-end against a live cloud account, which surfaced real operational issues that a purely theoretical design would not reveal - including provider-specific resource lifecycle constraints, drifting default values in managed-service APIs, and account-level resource limits - and the resolution of each is reflected in the final implementation and its documentation.

3. Objectives

1. To design a common, provider-agnostic logical architecture - three polyglot application tiers behind a load balancer, backed by a managed relational database and a managed cache - that can be specified once and mapped consistently onto multiple cloud platforms.
2. To implement this architecture as four independent, standalone OpenTofu root modules for DigitalOcean, AWS, GCP, and Azure, each expressed using that provider's own native networking, compute, load-balancing, and managed-database services rather than a lowest-common-denominator abstraction.
3. To identify and document the concrete differences between providers encountered during implementation, including networking/VPC semantics, load-balancer path-routing capability, managed-database provisioning behaviour, and minimum-cost or free-tier-eligible resource sizing.
4. To validate the framework through actual end-to-end deployment on a live cloud account, verifying that each application tier can reach its managed database and cache, and that the load balancer correctly health-checks and routes traffic to all three tiers.
5. To produce a maintainable, well-documented open-source starter codebase - with shared and per-provider documentation - usable as a reference implementation or teaching aid for future multi-cloud infrastructure work.

4. Scope of the Project

The scope of this project is deliberately bounded so that it stays focused on infrastructure provisioning rather than application development. The boundaries and limitations are summarized below:

- **Application scope** - Not a production application: the Node.js, Python, and PHP services deployed are minimal connectivity-verification endpoints, not production business logic.
- **Provider scope** - exactly four providers - DigitalOcean, AWS, GCP, and Azure - each deployed independently, one at a time; simultaneous cross-cloud deployment, replication, or automated failover between providers is not implemented.
- **Deployment scope** - each provider module targets a single region and a single environment; multi-region high availability is not implemented.
- **Compute model** - provisioned as virtual machines rather than containers or a managed Kubernetes service.
- **Networking** - load balancers are configured for HTTP only; TLS/HTTPS termination and custom-domain certificate management are explicitly out of scope.
- **Cost** - resource sizing is deliberately optimized for minimum cost and free-tier eligibility rather than production-scale capacity planning.
- **Security posture** - reflects a documented "starter" baseline (restricted SSH access, private database networking where the provider supports it), not a compliance-audited production configuration.

5. Literature Review

The literature on cloud computing establishes the foundational service and deployment models this project builds upon. Mell and Grance [2] formalize the now-standard definitions of Infrastructure-as-a-Service, Platform-as-a-Service, and the essential characteristics of cloud computing, while Armbrust et al. [1] provide an early, influential analysis of the economic and technical drivers behind cloud adoption. Erl, Mahmood, and Puttini [3] extend this into a broader treatment of cloud architecture patterns, including the networking and virtualization abstractions - such

as virtual private networks - that recur, in provider-specific form, throughout this project's implementation.

A second strand of literature addresses how software delivery practice evolved to manage infrastructure at this scale. Humble and Farley [4] popularized the discipline of Continuous Delivery, identifying manual, inconsistent environment provisioning as a primary source of deployment risk and arguing for automated, repeatable environment creation. Morris [5] and Brikman [6] build directly on this by documenting Infrastructure as Code (IaC) as a discipline in its own right, with Brikman's work specifically centred on Terraform - the tool from which OpenTofu, the technology used in this project, was forked. Newman [7] addresses the related architectural trend of decomposing monolithic systems into independently deployable, and often polyglot, services, which motivates this project's choice to run three different language runtimes as separate application tiers rather than a single unified service.

A third strand addresses multi-cloud strategy specifically. Petcu [8] surveys the technical expectations and early approaches to multi-cloud systems, noting the absence, at the time, of mature tooling to express one architecture across multiple providers - a gap that IaC tools with pluggable provider back-ends, such as Terraform/OpenTofu, were later built to close. Opara-Martins, Sahandi, and Tian [9] examine vendor lock-in from a business perspective, presenting empirical evidence that lock-in risk is a primary motivator for multi-cloud adoption - directly supporting this project's premise. Kratzke and Quint [10] conduct a systematic mapping study of cloud-native application design after a decade of cloud computing, observing that most published cloud-native architectures still target a single provider, with genuinely portable, multi-provider reference implementations comparatively rare. Mesbahi and Rahmani [11] survey load-balancing algorithms and architectures in cloud environments, providing the classification background against which this project's path-based routing design, and DigitalOcean's comparatively limited native load-balancer capability, are discussed.

Taken together, this literature establishes strong theoretical and conceptual grounding for cloud service models, IaC practice, and the motivations for multi-cloud adoption, but - consistent with Kratzke and Quint's [10] observation - offers comparatively few reproducible, openly documented reference implementations that actually provision an equivalent architecture across several major providers and validate it through a live

deployment. Existing IaC literature (e.g., Brikman [6]) is also predominantly single-provider in its worked examples. This project addresses that gap by contributing a concrete, tested, and openly documented four-provider implementation, including the provider-specific operational issues surfaced by live deployment.

For quick reference, Table 1 summarizes the works reviewed above.

Table 1: Summary of Reviewed Literature

Ref.	Author(s) & Year	Focus Area	Relevance to This Project
[1]	Armbrust et al. (2010)	Cloud computing economics & technical drivers	Motivates why organizations adopt cloud computing at all
[2]	Mell & Grance (2011)	Formal definition of cloud service/deployment models	Provides the IaaS/networking vocabulary used throughout
[3]	Erl, Mahmood & Puttini (2013)	Cloud architecture patterns	Background for VPC/VNet and virtualization concepts
[4]	Humble & Farley (2010)	Continuous Delivery	Origin of the case for automated, repeatable environments
[5]	Morris (2020)	Infrastructure as Code practice	Conceptual basis for the IaC approach used here
[6]	Brikman (2022)	Terraform in practice	Direct predecessor of OpenTofu, the tool used here
[7]	Newman (2021)	Microservices design	Basis for the polyglot, per-tier application design
[8]	Petcu (2013)	Multi-cloud expectations	Identifies the tooling gap this project helps close

[9]	Opara-Martins et al. (2016)	Vendor lock-in	Empirical motivation for a multi-cloud approach
[10]	Kratzke & Quint (2017)	Cloud-native application mapping study	Identifies the scarcity of multi-provider reference builds
[11]	Mesbahi & Rahmani (2016)	Load-balancing algorithms survey	Background for the path-based routing design

6. Proposed System / Implementation

The proposed system consists of a shared application layer and four independent, standalone OpenTofu root modules - one per cloud provider - that each provision the same logical architecture using that provider's native services. Figure 1 illustrates this common logical architecture: a user's request reaches a single load balancer, which routes it by URL path to one of the three application tiers, each of which can reach the same managed MySQL database and Redis-compatible cache.

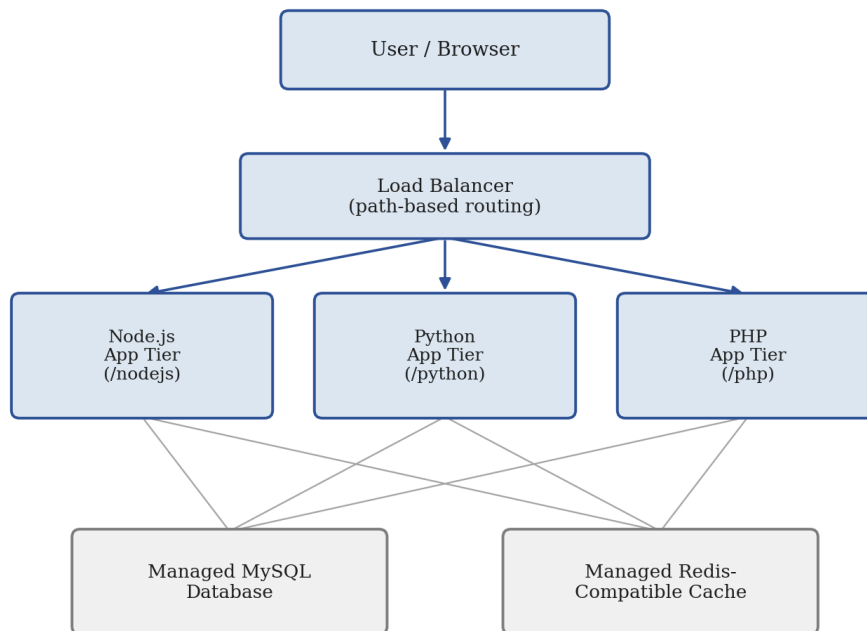


Figure 1: Common Logical Architecture of the Proposed Framework

Shared application layer

Each application tier (Node.js, Python, PHP) is a single-file, dependency-free HTTP service that reads its port and its managed MySQL/Redis endpoint from environment variables, and reports whether each backend is reachable over TCP. Configuration is injected at boot time via cloud-init, so the same application source is reused, unmodified, across all four providers.

Per-provider infrastructure modules

Each provider folder follows an identical file layout (network, compute, load balancer, database, cache, outputs) implemented with that platform's own resources: DigitalOcean Droplets, Managed MySQL, and a Valkey (Redis-compatible) cluster behind a Load Balancer; AWS EC2, RDS, and ElastiCache behind an Application Load Balancer; GCP Compute Engine, Cloud SQL, and Memorystore behind an external HTTP load balancer, connected to the managed services via Private Services Access; and Azure Virtual Machines, Azure Database for MySQL, and Azure Cache for Redis behind an Application Gateway.

Table 2 maps the common architecture in Figure 1 onto each provider's specific service names, making the differences in vocabulary and capability explicit.

Table 2: Multi-Cloud Service Mapping

Layer	DigitalOcean	AWS	GCP	Azure
Network	VPC	VPC	VPC (custom network)	Virtual Network (VNet)
Compute	Droplet	EC2 instance	Compute Engine VM	Virtual Machine
Load Balancer	Load Balancer + nginx edge proxy	Application Load Balancer	External HTTP Load Balancer	Application Gateway
Database	Managed MySQL	RDS (MySQL)	Cloud SQL (MySQL)	Azure Database for MySQL
Cache	Managed Valkey	ElastiCache (Redis)	Memorystore (Redis)	Azure Cache for Redis

Grouping	Project	Resource Group	Project (native)	Resource Group (native)
----------	---------	----------------	------------------	-------------------------

Path-based routing

All four providers expose the three application tiers through a single load balancer, routed by URL path (`/nodejs`, `/python`, `/php`). Three providers implement this natively in their load balancer's routing rules; DigitalOcean's Load Balancer cannot route by path on its own, so the DigitalOcean module adds a small `nginx` reverse-proxy instance behind the Load Balancer to perform the routing - a provider-specific workaround documented explicitly in that module.

Resource organization and cost

Resources are additionally grouped under a named container in each provider's console - a DigitalOcean Project, an AWS Resource Group, and the native project/resource-group constructs GCP and Azure already require - and default resource sizes are deliberately chosen to be the smallest or free-tier-eligible option each provider offers.

7. Tools and Technologies

- Infrastructure as Code: OpenTofu (HCL configuration language)
- Cloud platforms: DigitalOcean, Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure
- Managed compute: DigitalOcean Droplets, AWS EC2, GCP Compute Engine, Azure Virtual Machines
- Managed databases/cache: DigitalOcean Managed MySQL & Valkey, AWS RDS & ElastiCache, GCP Cloud SQL & Memorystore, Azure Database for MySQL & Azure Cache for Redis
- Load balancing: DigitalOcean Load Balancer, AWS Application Load Balancer, GCP External HTTP Load Balancer, Azure Application Gateway
- VM bootstrapping: `cloud-init`, `systemd`, `nginx` (DigitalOcean edge reverse proxy)
- Application runtimes: Node.js, Python 3, PHP
- Scripting and tooling: Bash, Python (tooling scripts)
- Provider CLIs: `doctl`, AWS CLI, `gcloud`, Azure CLI
- Version control and documentation: Git/GitHub, Markdown, Mermaid diagrams

8. Project Timeline

The timeline below follows the department's official Project Submission Schedule notice, which fixes four mandatory milestones - the synopsis, two monthly progress reports, and the final working project - rather than leaving submission dates to the student's discretion. Table 3 lists each milestone's due date alongside the development activities planned for that window, and Figure 2 presents the same schedule as a flowchart. The synopsis deadline (12 September 2026) is calculated as 10 days from the date of the notice; the two progress-report deadlines follow at one-month intervals thereafter, as specified in the notice; the final submission date is fixed only as "before the final examination" in the notice, so it is shown here as December 2026 pending the university's final examination timetable.

Table 3: Project Timeline per Official Submission Schedule

Milestone	Deliverable	Key Development Activities	Due Date
1	Project Synopsis	Literature review, requirement analysis, common multi-cloud architecture design	12 Sep 2026 (10 days from notice)
2	First-Month Progress Report	Shared application layer development; DigitalOcean & AWS module implementation	12 Oct 2026 (1 month after synopsis)
3	Second-Month Progress Report	GCP & Azure module implementation; load balancer & path-based routing integration	12 Nov 2026 (1 month after Milestone 2)
4	Final Working Project & Report	Live deployment testing & debugging; documentation, cost optimization, report writing	Dec 2026 (before final examinations)

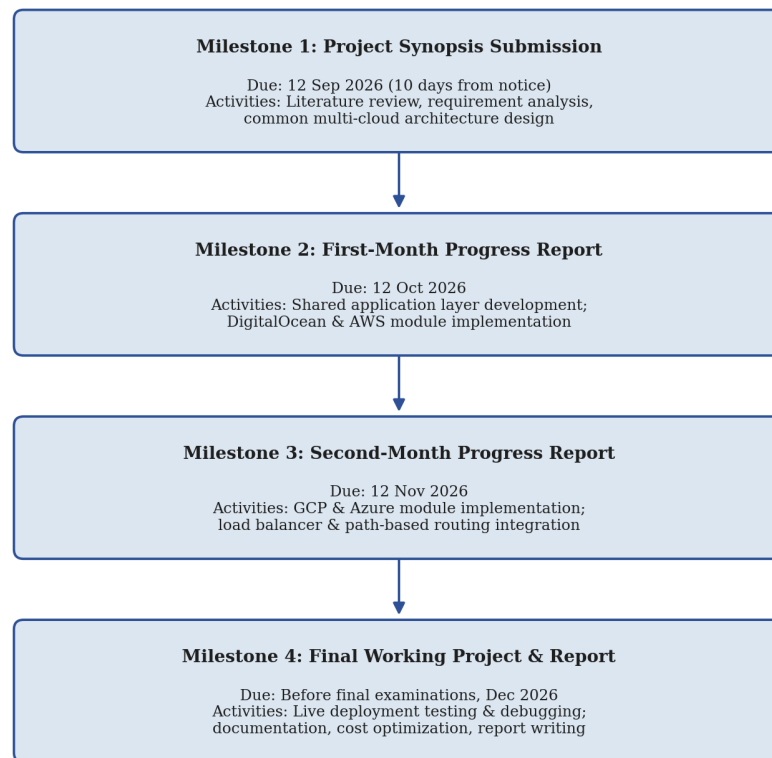


Figure 2: Project Timeline Flowchart

9. Expected Outcome

The expected outcome is a working, openly documented multi-cloud infrastructure-as-code repository comprising four independent, validated OpenTofu modules (DigitalOcean, AWS, GCP, Azure); a shared, provider-agnostic polyglot application layer; comprehensive root-level and per-provider documentation covering architecture, environment configuration, and cost; and a demonstrated, end-to-end successful deployment - verified on a live DigitalOcean account - in which the load balancer correctly health-checks and path-routes to all three application tiers, and each tier successfully reaches its managed MySQL database and Redis-compatible cache. A further outcome is a documented, comparative understanding of the four providers' infrastructure primitives, default costs, and free-tier eligibility.

10. Significance of the Project

The project has both practical and educational significance. Practically, it provides organizations and individual developers evaluating a cloud provider, planning a migration, or designing a genuinely portable deployment strategy with a working starting point that makes the real differences between providers explicit rather than theoretical, directly addressing the vendor lock-in risk documented by Opara-Martins et al. [9]. Educationally, it serves as a teaching resource for Infrastructure as Code and comparative cloud architecture, since the same logical design is expressed four times using four different sets of native primitives, alongside explicit documentation of why each provider's implementation differs. Because the architecture was validated through an actual live deployment rather than left as a theoretical design, the project's documentation also captures real operational issues - such as provider-specific resource lifecycle constraints and drifting managed-service defaults - that purely conceptual multi-cloud literature does not surface.

11. Future Scope

- Adding TLS/HTTPS termination with automated certificate provisioning (e.g., AWS ACM, Google-managed certificates, Azure Key Vault, Let's Encrypt).
- Supporting horizontal scaling of each application tier (multiple instances per tier, autoscaling groups) instead of a single instance behind the load balancer.
- Adding container-orchestrated variants of each module using each provider's managed Kubernetes service (EKS, GKE, AKS, DOKS).
- Implementing simultaneous multi-cloud deployment with cross-cloud traffic management and failover, rather than one provider at a time.
- Integrating CI/CD pipelines (e.g., GitHub Actions) to automate plan/apply/destroy workflows and enforce policy checks before deployment.
- Adding automated cost estimation and reporting (e.g., via Infracost) integrated into the development workflow.

12. Conclusion

This project set out to design and implement a multi-cloud infrastructure-as-code framework, using OpenTofu, that provisions an identical logical architecture - three polyglot application tiers behind a load balancer, backed by managed MySQL and Redis-compatible services - independently across DigitalOcean, AWS, GCP, and

Azure. The work confirms that while the logical architecture is portable in concept, its concrete realization must be adapted to each provider's native primitives, and that these adaptations - in networking, load-balancer capability, and managed-database behaviour - are significant enough to warrant explicit documentation rather than being abstracted away. Validating the framework through live deployment on a real cloud account, rather than relying solely on static configuration review, surfaced concrete operational issues that were resolved and fed back into the final implementation and its documentation. The resulting artifact is a practical, reusable, and openly documented foundation that demonstrates the role of Infrastructure as Code in managing the genuine complexity of multi-cloud environments, and is intended to serve as both a reference implementation and a teaching resource for future work in this area.

13. References

- [1] Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50-58.
- [2] Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing (NIST Special Publication 800-145). National Institute of Standards and Technology, U.S. Department of Commerce.
- [3] Erl, T., Mahmood, Z., & Puttini, R. (2013). *Cloud Computing: Concepts, Technology & Architecture*. Prentice Hall.
- [4] Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- [5] Morris, K. (2020). *Infrastructure as Code: Dynamic Systems for the Cloud Age* (2nd ed.). O'Reilly Media.
- [6] Brikman, Y. (2022). *Terraform: Up and Running: Writing Infrastructure as Code* (3rd ed.). O'Reilly Media.
- [7] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

- [8] Petcu, D. (2013). Multi-Cloud: expectations and current approaches. In Proceedings of the 2013 International Workshop on Multi-cloud Applications and Federated Clouds (pp. 1-6). ACM.
- [9] Opara-Martins, J., Sahandi, R., & Tian, F. (2016). Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective. *Journal of Cloud Computing*, 5(4), 1-18.
- [10] Kratzke, N., & Quint, P.-C. (2017). Understanding cloud-native applications after 10 years of cloud computing - a systematic mapping study. *Journal of Systems and Software*, 126, 1-16.
- [11] Mesbahi, M. R., & Rahmani, A. M. (2016). Load balancing in cloud computing: a state of the art survey. *International Journal of Modern Education and Computer Science*, 8(3), 64-78.